Quantum Machine Learning Methods for Semiconductor Wafer Defect Classification

Stephen Reagin

April 27, 2026

1 Introduction and Background

Quantum machine learning (QML) is an emerging field seeking to improve upon classical machine learning by leveraging the computational properties of quantum systems. A key motivation is that a quantum system of n qubits can process information in a Hilbert space of dimension 2^n, which grows exponentially with qubit count. Quantum *feature maps* can encode classical data into this high-dimensional space, potentially capturing complex patterns via feature interactions that are classically intractable to represent explicitly [1].

A central tool in both classical and quantum ML is the *kernel method*. Given a dataset, a kernel function $K(x_i, x_{i'})$ computes the inner product between data points in some feature space, enabling algorithms such as Support Vector Machines (SVMs) to find non-linear decision boundaries without explicitly constructing that space, in a generalization known as the *kernel trick* [2]. For example, the widely-employed Support Vector Machine model can use one of several kernels to draw a series of best-fitting hyperplane decision boundaries via the relation $f(x) = \beta_0 + \sum_{i \in S} \alpha_i K(x, x_i)$ [3]:

- Linear kernel: $K(x_i, x_{i'}) = \sum_{j=1}^{p} x_{ij} x_{i'j}$
- Polynomial kernel: $K(x_i, x_{i'}) = (1 + \sum_{j=1}^{p} x_{ij} x_{i'j})^d$
- Radial kernel: $K(x_i, x_{i'}) = \exp(-\gamma \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2)$

Classically, computing a kernel matrix over n samples requires $\mathcal{O}(n^2)$ evaluations, which becomes a bottleneck for large datasets. A QML approach replaces the classical kernels with a quantum kernel $K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$, where $|\phi(x)\rangle$ is the quantum state produced by a parametrized feature map circuit. The inner product is computed by measuring the overlap between two quantum states, which can be estimated efficiently on quantum hardware even when the underlying Hilbert space is exponentially large [1].

This project applies both classical and quantum kernel methods to the supervised classification problem of detecting semiconductor wafer defects to benchmark the performance of QML methods against standard classical models. Defects introduced during semiconductor wafer fabrication can reduce yield, making rapid and accurate defect detection essential for quality control. Automated defect classification systems are increasingly using machine learning techniques to analyze wafer maps and identify patterns of faulty dies [4]. QML algorithms have been proposed to exploit high-dimensional quantum feature spaces and parameterized quantum circuits for data analysis [5][6], and while QML methods have shown promise on small synthetic datasets, their performance on realistic industrial datasets remains largely unexplored.

2 Models and Methods

2.1 Dataset

Classical and quantum machine learning models were trained on the MixedWM38 WaferMap dataset [7][8], a publicly available dataset of semiconductor wafer maps used in defect detection research. This dataset contains 38,000 wafer images represented as 52×52 grids, with each pixel corresponding to

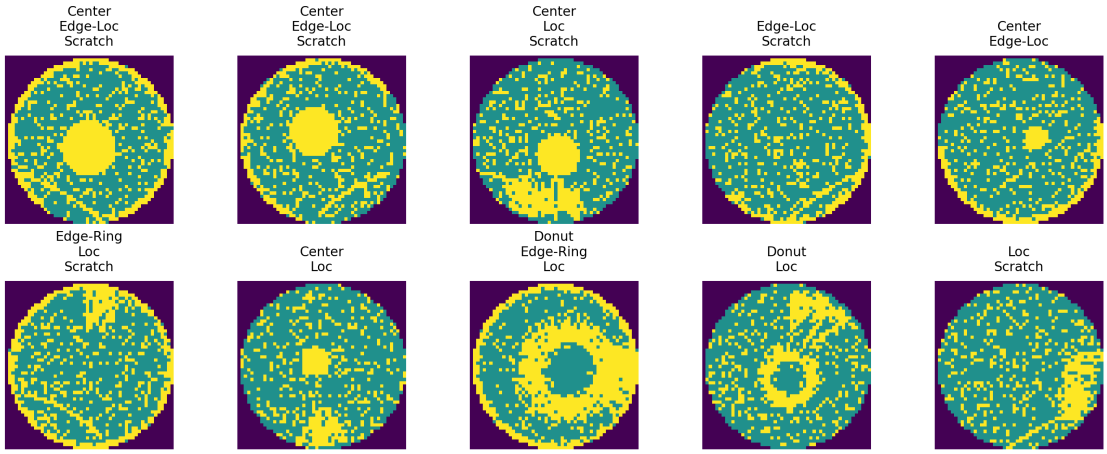


Figure 1: Examples of MixedWM38 semiconductor wafer images. Each image is 52 x 52 pixels, and each pixel represents a die that is functional (green), defective (yellow), or absent (purple). The macroscopic defect patterns are labeled above each wafer image.

a die on the wafer. Pixel values indicate whether the die is absent, functional, or defective, as seen in Figure 1. Many wafers exhibit spatial patterns of failure which can take several characteristic forms, including: (1) center defects, (2) donut, (3) edge-localized, (4) edge-ring, (5) localized, (6) nearly full wafer failure, (7) scratches, and (8) random. Each wafer is labeled using an 8-dimensional Boolean vector indicating which defect patterns are present.

2.2 Methodology

The basic methodology is:

1. Flatten and process the data so that it can conforms to the scikit-learn library of standard machine learning models.
2. Reduce the data dimensionality down from $52 \times 52 = 2704$ down to approximately a dozen dimensions.
3. Extract a representative 20% sample of the dataset to further reduce the computational resources to simulate a quantum circuit.
4. Train and benchmark:
   - Classical support vector machine (SVC) classifier
   - Quantum SVC classifier (QSVC) using Qiskit Machine Learning
   - QSVC using the Pennylane library
   - Variational quantum classifier (VQC) using Qiskit Machine Learning
   - VQC using Pennylane
5. Compare results.

Because each wafer image contains $52 \times 52 = 2704$ pixels, directly encoding the raw data into a quantum circuit is not feasible with current quantum hardware or classical simulators. Principal Component Analysis (PCA) is applied to reduce the feature dimensionality to n components, where n is swept from 1 to 30 for the classical SVM and from 2 to 11 for the QSVC and VQC, allowing the effect of circuit width on model performance to be studied systematically. Prior to PCA, pixel values are standardized using z-score normalization to zero mean and unit variance, ensuring PCA components are not dominated by high-variance but low-information pixel regions.

Due to the $\mathcal{O}(n^2)$ cost of computing the quantum kernel matrix, training the QSVC and VQC on the full dataset is computationally intractable on classical hardware. A stratified subsample of 20% of

the full dataset is used, preserving class proportions across all eight defect types. This subsample is then split into 80% training and 20% test sets.

2.3 Models

1. **Classical SVC baseline:** A classical SVM with a radial kernel (`'rbf'`) is trained on the PCA-reduced features using scikit-learn. The regularization parameter is set to `C=1` with `gamma='scale'`, and `class_weight='balanced'` to account for class imbalance across defect types. The sweep of n PCA dimensions goes from 1 to 30 dimensions.

2. **Quantum support vector classifier (QSVC):** A quantum kernel is constructed using the ZZ feature map with n qubits and 2 repetitions, implemented in Qiskit Machine Learning [9]. The quantum kernel is defined as $K(x_i, x_j) = |\langle\phi(x_i)|\phi(x_j)\rangle|^2$, where $|\phi(x)\rangle$ is the quantum state produced by the feature map circuit. The kernel is computed using the `FidelityStatevectorKernel`, which is a non-noisy "perfect" quantum statevector simulator. Then it is passed to a classical SVM optimizer where the learning occurs during SVM optimization while the feature map circuit remains fixed.

3. **Variational quantum classifier (VQC):** A parameterized quantum circuit is implemented using Qiskit Machine Learning [10], consisting of the ZZ feature map for data encoding followed by a `RealAmplitudes` ansatz with 2 repetitions as the trainable component. Circuit parameters are optimized using the COBYLA optimizer with a maximum of 100 iterations, minimizing a cross-entropy classification loss. The `StatevectorSampler` is used for circuit execution.

2.4 Evaluation Metrics

		Predicted	
		0	**1**
Actual	**0**	TN	FP
	1	FN	TP

		Predicted	
		0	**1**
Actual	**0**	138	12
	1	15	35

Table 1: Confusion matrix structure (left) and an example of QSVC predictions (right).

All models are evaluated on four common classification metrics derived from the well-studied *confusion matrix* seen in Table 1, where TN = True Negatives, TP = True Positives, FN = False Negatives, and FP = False Positives. The four classification metrics include:

- **Accuracy** $\equiv \frac{TP+TN}{TP+TN+FP+FN}$

  The fraction of all predictions, both defective and non-defective, that the model classified correctly.

- **Precision** $\equiv \frac{TP}{TP+FP}$

  Of all wafers the model predicted as defective, what fraction were truly defective.

- **Recall** $\equiv \frac{TP}{TP+FN}$

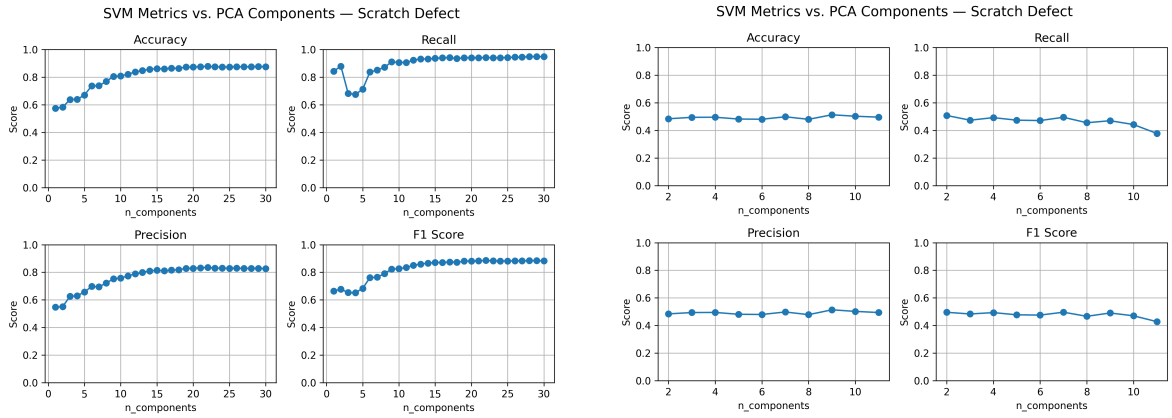
  Of all wafers that were truly defective, what fraction the model successfully predicted as being defective.

- **F1 score** $\equiv \frac{2TP}{2TP+FP+FN}$

  The harmonic mean of precision and recall, balancing both metrics into a single score that is particularly informative under class imbalance.

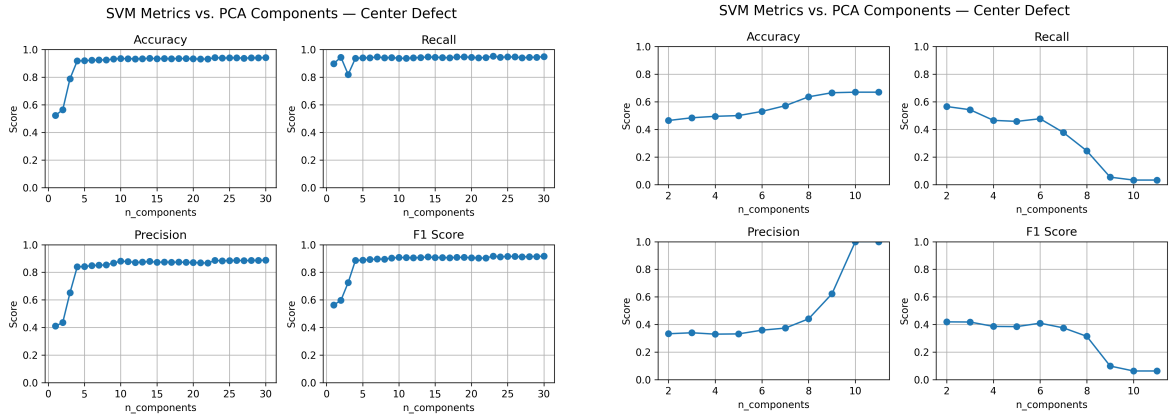
3 Results

This section is still in progress. The early results are:



Classical SVM QSVC

Figure 2: Classification metrics vs. n number of PCA components for the Scratch defect, Classical SVM (left) vs. QSVC (right).



Classical SVM QSVC

Figure 3: Classification metrics vs. n number of PCA components for the Center defect, Classical SVM (left) vs. QSVC (right)

- For classical SVM: all metrics tend to improve as the n number of reduced PCA dimensions is increased. Furthermore, the improvements are very significant when increasing from low dimensions, e.g. going from 2 to 5 dimensions results in a significant scoring improvement. Going from 10 to 15 dimensions has a much more modest improvement, and scores start leveling off into a steady state at around 15 dimensions, as seen in Figure 2.

- For classical SVM: the speed was not carefully benchmarked, but it takes fewer than ten minutes to train 30 different models of differing dimensions on the 8 different target datasets. When the training data is further stratified via sampling, the total runtime can be reduced to about two minutes.

- For QSVC: metric results do not always meaningfully increase as the number of PCA dimensions increases. As seen in Figures 2 and 3, sometimes the metrics remain flat with increasing dimensions, and sometimes they rise or fall dramatically as a function of dimension.

- For QSVC: each model iteration takes roughly 2-3 minutes to train when n is low, and 5-10 minutes or more as n approaches 12 and higher. The total training time for this model sequence, including 8 defect target datasets, and sweeping from $n = 2$ to $n = 11$, was approximately **6 hours.**

- For VQC: I only ran one iteration with $n = 2$ dimensions and it took 15 minutes to run. Running

a full loop for comparison against QSVC and classical SVM will likely take very many hours of training time.

- For the Pennylane versions of QSVC and VQC: these models were never trained because the Pennylane kernel implementation (a necessary step before any model training occurs) was taking far too long, I believe roughly 30 minutes is when I interrupted the program so that resources could be allocated to the Qiskit versions that worked much more quickly.

4 Discussion and Conclusions

This section is still in progress. The classical SVM demonstrated consistent and predictable improvement as the number of PCA components increased, plateauing at approximately 15 dimensions, consistent with well-established RBF kernel behavior [3]. The QSVC exhibited markedly different behavior, with classification metrics fluctuating irregularly as a function of n rather than improving monotonically. This instability is consistent with findings in the QML literature, where quantum kernel methods have been shown to be sensitive to the choice of feature map and encoding strategy [5]. Overall, the classical SVM outperformed the QSVC across most defect types and PCA configurations evaluated.

The VQC and PennyLane implementations were not fully evaluated due to prohibitive simulation runtimes. A single VQC iteration at $n = 2$ dimensions required approximately 15 minutes, and the QSVC sweep across 8 defect types and 10 PCA configurations required approximately 6 hours. This reflects a fundamental limitation of classical statevector simulation, where the exponential state space that gives quantum computing its theoretical advantage becomes a computational liability when simulated classically [6]. Consequently, no conclusions can be drawn about performance on real quantum hardware from these results alone.

The computational overhead observed here underscores a well-documented challenge in near-term QML: the path to practical quantum advantage for classification tasks remains unclear [6]. Future work could address these limitations by executing models on real quantum hardware, exploring alternative feature maps, or investigating trainable quantum kernels. In conclusion, classical SVM outperformed the quantum classifiers evaluated under the constraints of classical simulation, consistent with the broader QML literature on near-term devices.

AI Disclaimer: ChatGPT and Gemini were used to organize research notes into an early draft of this report, and Claude to develop Python code for reading MixedWM348 data.

References

[1] C. Conti, *Quantum Machine Learning: Thinking and Exploration in Neural Network Models for Quantum Science and Quantum Computing* (Springer International Publishing, 2023).

[2] M. Kuhn and K. Johnson, *Applied Predictive Modeling* (Springer, 2013).

[3] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2nd ed. (Springer, 2021).

[4] Y. Kim, J.-S. Lee, and J.-H. Lee, IEEE Transactions on Semiconductor Manufacturing **36**, 476 (2023).

[5] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, Nature **567**, 209 (2019).

[6] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, Nature **549**, 195 (2017).

[7] J. Wang, C. Xu, Z. Yang, J. Zhang, and X. Li, IEEE Transactions on Semiconductor Manufacturing **33**, 587 (2020).

[8] GitHub - Junliangwangdhu/WaferMap — github.com, `https://github.com/Junliangwangdhu/WaferMap`, [Accessed 16-03-2026].

[9] Qiskit Community, Qsvc — qiskit machine learning documentation, https://qiskit-community.github.io/qiskit-machine-learning/stubs/qiskit_machine_learning.algorithms.QSVC.html (2024).

[10] Qiskit Community, Vqc — qiskit machine learning documentation, https://qiskit-community.github.io/qiskit-machine-learning/stubs/qiskit_machine_learning.algorithms.VQC.html (2024).